



Heaps e Filas de Prioridade

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Propriedades dos Heaps

Operações

Filas de Prioridade

Heap Sort

1. Propriedades dos heaps (min/max, array)



2. Operações: *heapify*, insert, extract



3. Filas de prioridade



4. Heap Sort

Propriedades dos Heaps

Problema:

Precisamos repetidamente do *menor* (ou *maior*) elemento de um conjunto que muda com o tempo: inserções e remoções intercaladas.

Lista ordenada: inserção custosa ($O(n)$).
Lista desordenada: busca do mínimo custosa ($O(n)$).

Solução:

O **heap** mantém o extremo sempre no topo e equilibra os custos:

- inserção em $O(\log n)$
- remoção do extremo em $O(\log n)$
- acesso ao extremo em $O(1)$

Definição

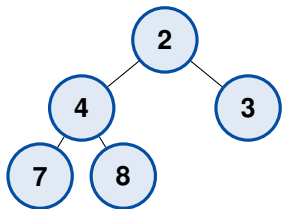
Um **heap binário** é uma árvore binária *quase completa* que satisfaz a **propriedade de heap**:

- **Min-heap**: todo nó é $\leq$ seus filhos $\Rightarrow$ o *mínimo* fica na raiz.
- **Max-heap**: todo nó é $\geq$ seus filhos $\Rightarrow$ o *máximo* fica na raiz.

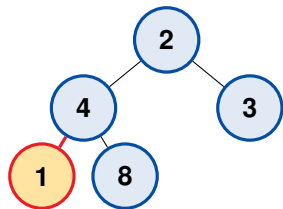
Intuição

Não há ordenação total entre irmãos — apenas a relação pai/filho importa. Por isso é mais barato de manter que uma estrutura totalmente ordenada.

Exemplo: isto é um min-heap?



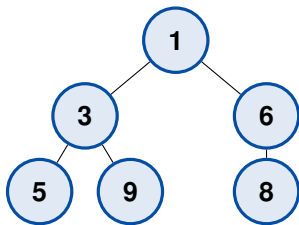
válido: todo pai $\leq$ filhos ✓



1 < 4: viola! — inválido

Verifique aresta por aresta: pai $\leq$ filho (min-heap).

Árvore quase completa $\Rightarrow$ array



Min-heap como árvore

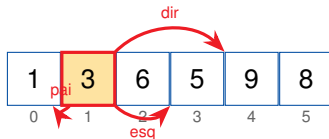
1	3	6	5	9	8
0	1	2	3	4	5

Índices (base 0)

$$\text{pai}(i) = \lfloor (i - 1) / 2 \rfloor$$

$$\text{esq}(i) = 2i + 1 \quad \text{dir}(i) = 2i + 2$$

As fórmulas de índice viram *contas concretas*. Tomemos $i = 1$ (valor **3**) e localizemos seu pai e seus dois filhos no array.



Contas para $i = 1$

$$\text{esq}(1) = 2 \cdot 1 + 1 = 3 \Rightarrow A[3] = 5$$

$$\text{dir}(1) = 2 \cdot 1 + 2 = 4 \Rightarrow A[4] = 9$$

$$\text{pai}(1) = \lfloor (1 - 1) / 2 \rfloor = 0 \Rightarrow A[0] = 1$$

O que aprendemos

- Heap = árvore binária quase completa + propriedade de heap.
- Min-heap mantém o mínimo na raiz; max-heap, o máximo.
- A estrutura é armazenada de forma compacta em um **array**, sem ponteiros.
- Navegação por aritmética de índices: $2i + 1$, $2i + 2$, $\lfloor (i - 1)/2 \rfloor$.

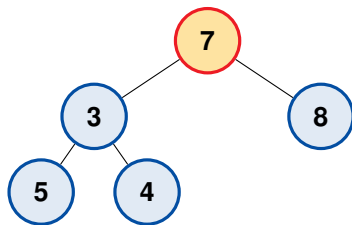
Operações

- **sift-down** (*heapify*): empurra um nó para baixo até restaurar a propriedade.
- **sift-up**: empurra um nó para cima após uma inserção.
- **insert**: adiciona ao fim + sift-up.
- **extract-min**: remove a raiz, traz o último ao topo + sift-down.
- **build-heap**: transforma um array qualquer em heap.

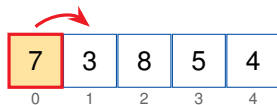
Complexidades

Operação	Custo
acesso ao mínimo	$O(1)$
insert	$O(\log n)$
extract-min	$O(\log n)$
build-heap	$O(n)$

Sift-down: passo 1 de 3

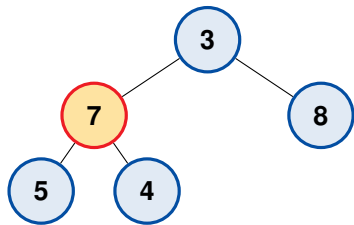


Raiz 7 viola a propriedade de min-heap

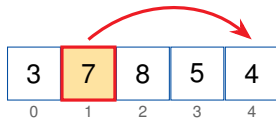


Menor filho = 3 (idx 1). Como $7 > 3$, troca raiz $\leftrightarrow$ idx 1.

Sift-down: passo 2 de 3

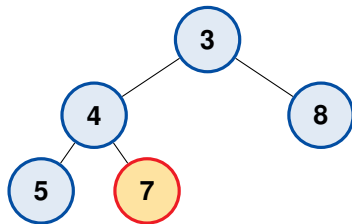


7 desceu para idx 1; compara com 5 e 4



Menor filho = 4 (idx 4). Como $7 > 4$, troca idx 1
 $\leftrightarrow$ idx 4.

Sift-down: passo 3 de 3 (fim)



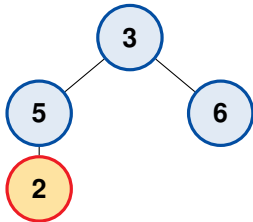
7 chegou em idx 4, uma folha

Caminho percorrido $\leq$ altura $\Rightarrow$ custo $O(\log n)$.

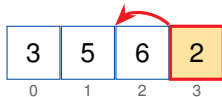


7 chegou a uma folha: propriedade de min-heap restaurada.

Insere na próxima posição livre (fim) e sobe enquanto for menor que o pai.

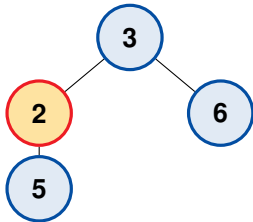


2 anexado em idx 3 (filho esq. de idx 1)

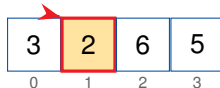


Pai de idx 3 é idx 1 (valor 5). Como $2 < 5$, sobe: troca idx 3 $\leftrightarrow$ idx 1.

O 2 subiu uma vez; continua comparando com o novo pai.



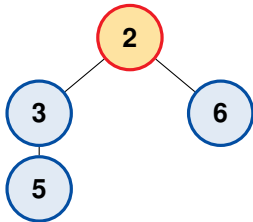
2 agora em idx 1



Pai de idx 1 é idx 0 (valor 3). Como $2 < 3$, sobe: troca
idx 1 $\leftrightarrow$ idx 0.

Insert (sift-up): passo 3 de 3 (fim)

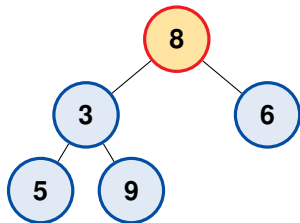
O 2 alcançou a raiz: não há pai, então o sift-up para.



2 é a nova raiz (idx 0)



2 virou a nova raiz. Custo $O(\log n)$.



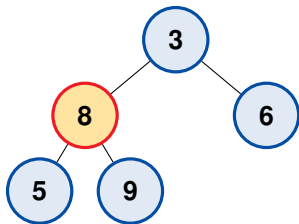
Último (8) sobe ao topo



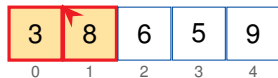
Ideia: o mínimo está na raiz.

Removemos a raiz (2) e trazemos o **último** elemento (8, idx 5) ao topo. $A = [8, 3, 6, 5, 9]$, tamanho 5.

retorna 2



8 desce por sift-down

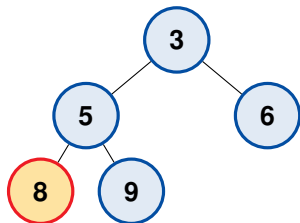


Filhos da raiz: 3 (idx 1) e 6 (idx 2); menor filho = 3.

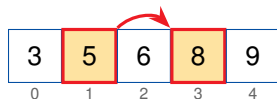
Como $8 > 3$, troca idx 0 $\leftrightarrow$ idx 1.

$A = [3, 8, 6, 5, 9]$.

Extract-min: passo 3 de 3 (fim)



8 vira folha $\Rightarrow$ para



Filhos de 8: 5 (idx 3) e 9 (idx 4); menor filho = 5.

Como $8 > 5$, troca idx 1 $\leftrightarrow$ idx 3; depois 8 vira
folha $\rightarrow$ para. $A = [3, 5, 6, 8, 9]$.

Retornou 2; heap restaurado em $O(\log n)$.

```
1  def extract_min(heap):
2      minimo = heap[0]           # raiz = menor
3      ultimo = heap.pop()       # remove o ultimo
4      if heap:
5          heap[0] = ultimo       # ultimo vai ao topo
6          sift_down(heap, 0)     # restaura propriedade
7      return minimo             # custo total  $O(\log n)$ 
```

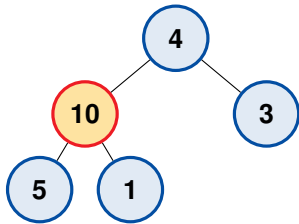
Aplicar *sift-down* de baixo para cima, começando no último nó interno $\lfloor n/2 \rfloor - 1$ até a raiz.

A maioria dos nós está perto das folhas e percorre pouca altura. A soma $\sum_h \frac{n}{2^{h+1}} \cdot h$ converge para $O(n)$.

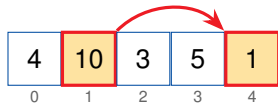
Por que importa

Construir o heap de uma vez é *linear* — mais barato que inserir os n elementos um a um ($O(n \log n)$).

Aplicamos *sift-down* nos nós internos, do último (idx 1) até a raiz; folhas são ignoradas.



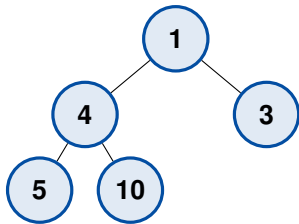
Foco: idx 1; filhos 5 (idx 3) e 1 (idx 4)



menor filho = 1. $10 > 1$: **troca idx 1** $\leftrightarrow$ **idx 4**.

$[4, 10, 3, 5, 1] \rightarrow [4, 1, 3, 5, 10]$

Agora *sift-down* na raiz (idx 0).



Min-heap final: raiz = 1 é o mínimo



$4 > 1$: troca idx 0 $\leftrightarrow$ idx 1; depois 4 (em idx 1) $\leq$ filhos 5 e 10 $\Rightarrow$ para.

$[4, 1, 3, 5, 10] \rightarrow [1, 4, 3, 5, 10]$

build-heap custa $O(n)$, não $O(n \log n)$.

O que aprendemos

- *sift-down* e *sift-up* são os blocos básicos, ambos $O(\log n)$.
- insert = anexa ao fim + sift-up; extract = troca topo/fim + sift-down.
- build-heap em $O(n)$ aplicando sift-down de baixo para cima.

Filas de Prioridade

Problema:

Atender elementos por *prioridade*, não por ordem de chegada: escalonamento de processos, emergências de um hospital, eventos de uma simulação.

Solução:

Uma **fila de prioridade** é um TAD com operações `insert(x)` e `extract-min/max()`.

O **heap** é a implementação eficiente padrão.

Definição (TAD)

Coleção de elementos com chave de prioridade que suporta:

- `insert(x)` — adiciona um elemento;
- `peek()` — consulta o de maior prioridade;
- `extract()` — remove e retorna o de maior prioridade.

Aplicação clássica

Algoritmo de **Dijkstra**: a cada passo extrai o vértice de menor distância — exatamente um `extract-min`.

Exemplo: extração em ordem de prioridade

Min-heap com chaves $\{2, 3, 6, 5, 9, 8\}$: cada `extract-min` entrega sempre a de maior prioridade (menor chave).



↓ `extract-min` repetido



saída ordenada (crescente)

Cada `extract-min` custa $O(\log n)$; a fila entrega o de maior prioridade primeiro.
Esta ideia é exatamente a base do Heap Sort (a seguir).

O que aprendemos

- Fila de prioridade é um TAD; heap é sua implementação eficiente.
- `insert` e `extract` custam $O(\log n)$; `peek`, $O(1)$.
- Base de algoritmos como Dijkstra, Prim e escalonadores.

Heap Sort

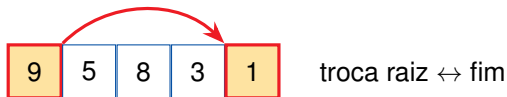
Problema:

Ordenar n elementos com garantia de $O(n \log n)$ no *pior caso* e **sem memória extra**.

Solução:

Construir um max-heap e extrair o máximo repetidamente, colocando-o no fim do próprio array — ordenação **in-place**.

1. **build-heap** (max-heap) sobre o array — $O(n)$.
2. Troca a raiz (máximo) com o último elemento do heap.
3. Reduz o tamanho do heap em 1 e aplica *sift-down* na raiz.
4. Repete até o heap ter 1 elemento.



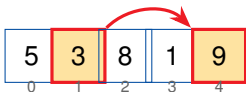
Após a troca, **9** fica fixo na posição final e o heap encolhe.

Heap Sort: construindo o max-heap

build-heap (max) processa nós internos idx 1 e 0.

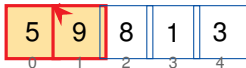
passo a (idx1=3): filhos 1, 9; maior 9; $3 < 9 \Rightarrow$ troca

idx1 $\leftrightarrow$ idx4

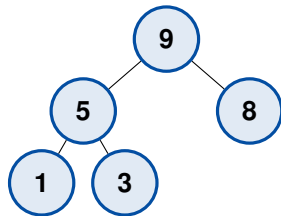


passo b (idx0=5): filhos 9, 8; maior 9; $5 < 9 \Rightarrow$ troca

idx0 $\leftrightarrow$ idx1; depois $5 \geq \{1, 3\}$ para



max-heap pronto: [9, 5, 8, 1, 3] (raiz = máximo)



Árvore do max-heap:

raiz 9; $\geq$ todos os filhos

Heap Sort: extraindo o máximo

Cada linha: troca raiz $\leftrightarrow$ fim e *heapify*; sufixo ordenado em azul (| separa heap | ordenado).

9	5	8	1	3
3	5	8	1	9
1	5	3	8	9
3	1	5	8	9
1	3	5	8	9

troca 9 $\leftrightarrow$ idx4

heapify $\rightarrow$ [8, 5, 3, 1 | 9], troca 8 $\leftrightarrow$ idx3

heapify $\rightarrow$ [5, 1, 3 | 8, 9], troca 5 $\leftrightarrow$ idx2

heapify $\rightarrow$ [3, 1 | 5, 8, 9], troca 3 $\leftrightarrow$ idx1

fim $\Rightarrow$ [1, 3, 5, 8, 9]

n extrações $\times O(\log n) = O(n \log n)$, in-place.

Análise

- build-heap: $O(n)$
- n extrações $\times O(\log n)$: $O(n \log n)$
- **Total:** $O(n \log n)$ no pior caso
- Espaço extra: $O(1)$ (in-place)

Atenção

Não é estável; na prática perde para o quicksort por constantes e localidade de cache.

O que aprendemos

- Ordena in-place em $O(n \log n)$ no pior caso.
- Reaproveita build-heap + extract-max repetido.
- Garante o limite assintótico, mas não é estável.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Robert W. Floyd.
Algorithm 245: Treesort 3.
Communications of the ACM, 7(12):701, 1964.



Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley, 4 edition, 2011.



J. W. J. Williams.
Algorithm 232: Heapsort.
Communications of the ACM, 7(6):347–348, 1964.



Nivio Ziviani.
Projeto de Algoritmos: com Implementações em Pascal e C.
Cengage Learning, 3 edition, 2011.